Sandia National Laboratories

Exceptional service in the national interest

SANGO: STRUCTURED ABSTRACTIONS FOR NETWORK GROUP ORGANIZATION

Felix Wang, Yang Ho, Michael C. Krygier,
Fred Rothganger, James B. Aimone

Neural Exploration and Research Laboratory (NERL)

Cognitive and Emerging Computing
Scalable AI Models & Algorithms

ICONS 2026, Tutorial

U.S. DEPARTMENT OF ENERGY | NNSA National Nuclear Security Administration

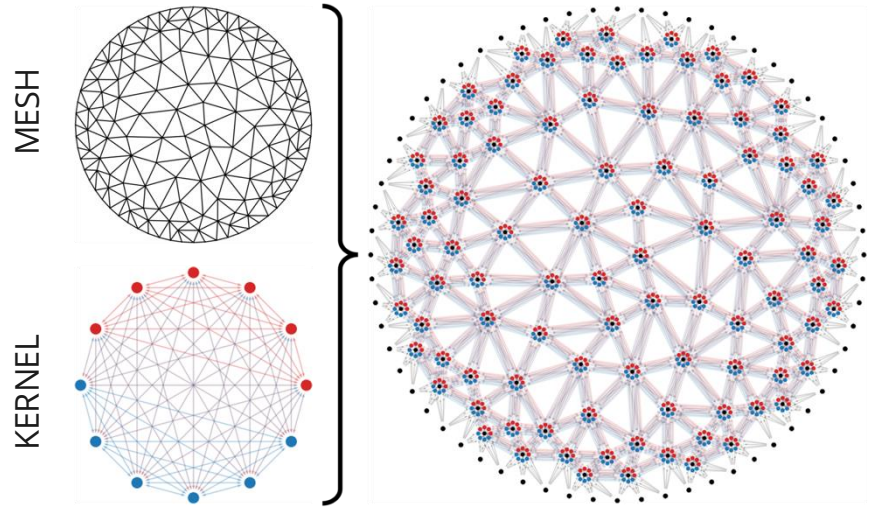
Sandia National Laboratories is a multimission laboratory managed and operated by National Technology and Engineering Solutions of Sandia LLC, a wholly owned subsidiary of Honeywell International Inc. for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525.

SAND2026-24730C

INTRODUCTION AND MOTIVATION

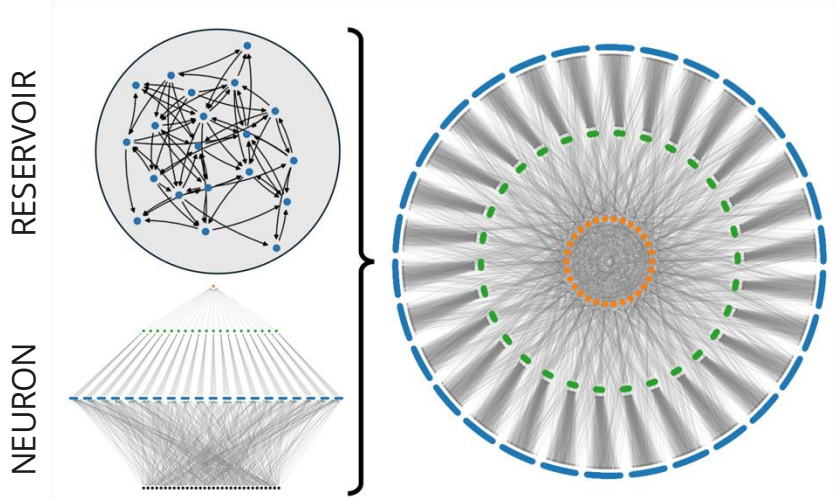
- Sango is a compositional spiking neural network domain-specific language
  - As neuromorphic algorithms become more complex, **managing its structure** becomes increasingly important

Neural Finite-Element Method

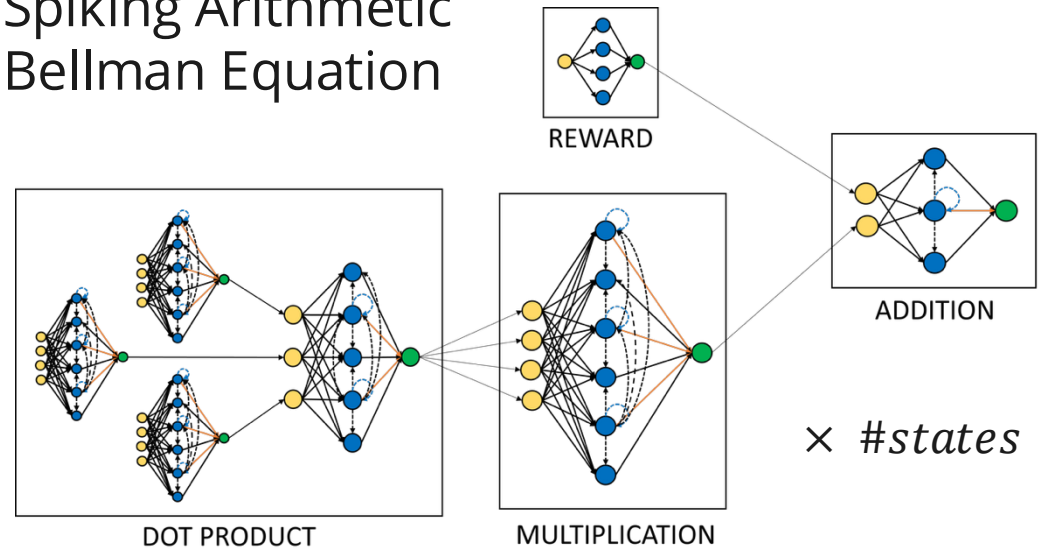


https://github.com/sandialabs/Sango

Dendritic Reservoir Network



Spiking Arithmetic Bellman Equation



DESIGN GOALS

- How can we make it easier to develop spiking neural networks?
  - Minimizing boilerplate, and the code that needs to be (re)written
    - Main objective of domain-specific language, providing **high-level abstractions**
    - Taking inspiration from PyTorch, adopting a **user-friendly front-end** API in Python
  - Focus on developing compositional, hierarchical, and recurrent networks
    - Taking inspiration from Fugu, allowing for **procedural generation** and **code reuse**
  - General, extensible, hardware agnostic, interoperable with other tools
    - Focus on a **straightforward intermediate representation**, easy to translate
    - Flexibility in neuron and synapse models, leave implementation to the backends

BASIC SYNTAX

```
net = Network()                          # instantiate network
net.layer = [NodeGroup(LIF(), 32),       # add a list of node groups
             NodeGroup(LIF(), 10)]
net.dense = EdgeGroup(net.layer[0],      # add an edge group connecting
                      net.layer[1],      # the node groups
                      PSP(), edges=...)
net.inp = NodePort()                     # add an input port (not linked)
net.out = NodeList(net.layer[1][::2]) # add an output list (slicing)

net.build()       # build network topology
print(net)        # print the topological components
```

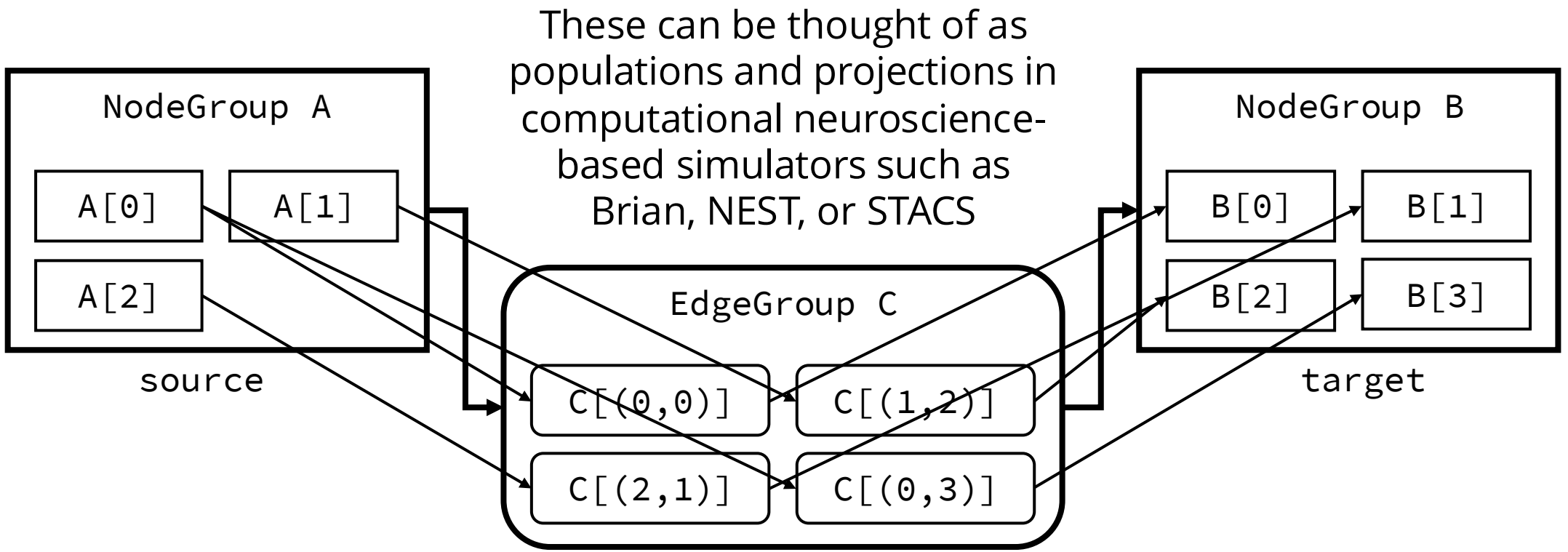
```
(node) layer[0]
(node) layer[1]
(edge) dense: (node) layer[0] -> (node) layer[1]
(port) inp <- (no link)
(list) out
```

HIERARCHICAL MENTAL MODEL

- **Filesystem hierarchy**: organize complex networks analogous to a directory listing of **folders** (networks themselves), **files** (groups of elements), and their **contents** (nodes and edges)
  - **`Network`**: the main container that is built out of high-level network components (including other networks)
  - **`NodeGroup`**: a group of instantiated nodes sharing the same node model (e.g. a population of neurons)
  - **`EdgeGroup`**: a group of instantiated edges between two sets of nodes (e.g. a projection of synapses)

BASIC NODE AND EDGE GROUPS

Node and edge groups are list-based containers for nodes and edges




```
A = NodeGroup(LIF(), 3)
B = NodeGroup(LIF(), 4)
C = EdgeGroup(A, B, PSP(), [(0,0), (1,2), (2,1), (0,3)])
```

NODE AND EDGE MODELS

- Node and edge models are defined as Python **`dataclasses`**
  - Model attributes may be assigned by default, in bulk, or individually
  - A model registry is used to interface with its associated backend

- Sango provides a few base models (supported by all backends)
  - **`IN`** – spike generator input model (spike times)
  - **`LIF`** – leaky integrate-and-fire neuron model (voltage, threshold, leak)
  - **`PSP`** – post-synaptic potential synapse model (weight, delay)

MODEL SYNTAX

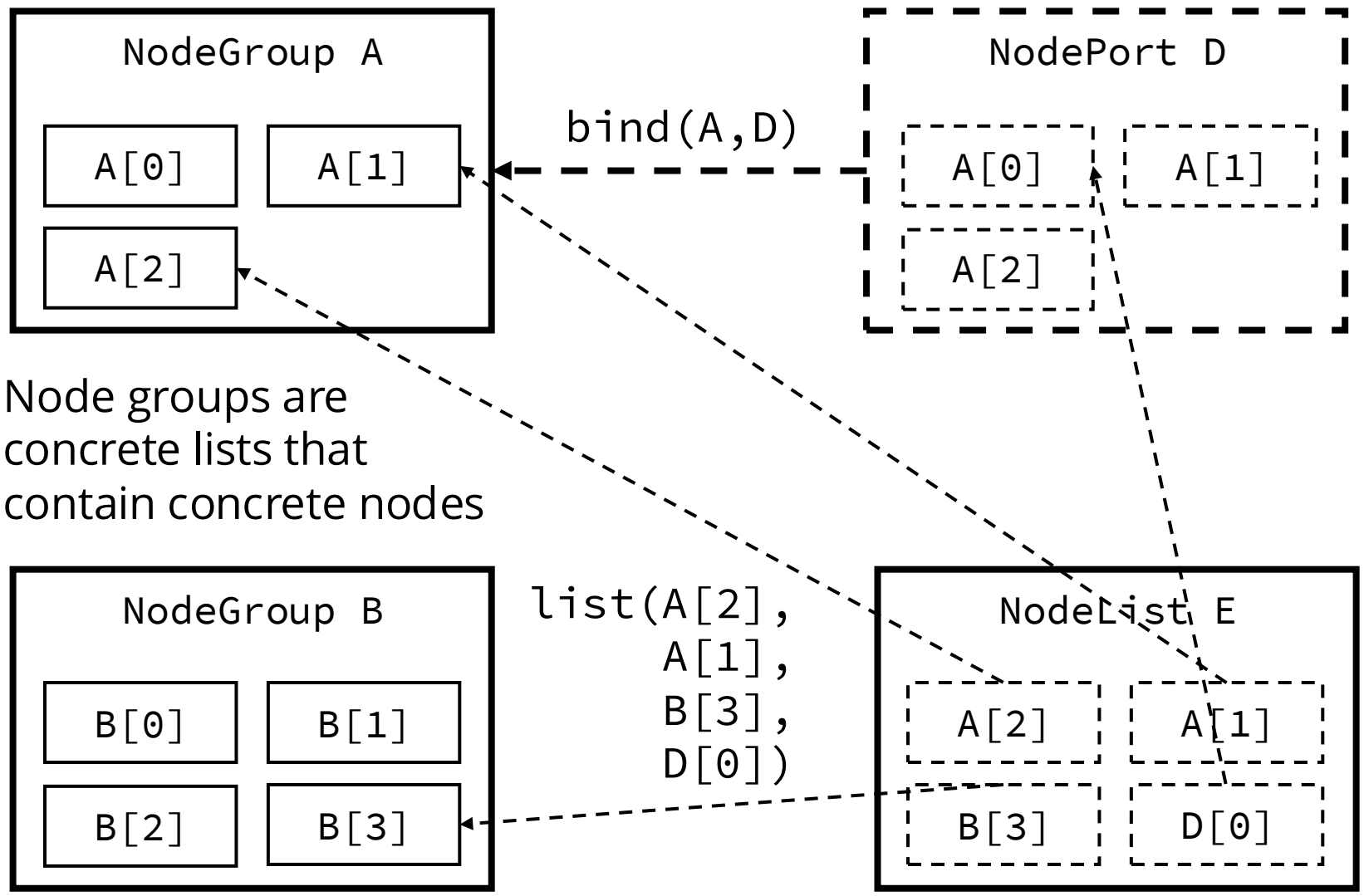
```
@dataclass
class MyLIF(Neuron):
    model: str = 'MyLIF'      # model name
    voltage: float = 0.0      # individual parameter
    threshold: float = 1.0    #   (may be unique per instance)
    reset: float = (0.0,)     # shared parameter (tuple format)
    leak: float = 1.0         #   (across all instances in group)

pop = NodeGroup(MyLIF(threshold=0.9),    # model and parameter defaults
                size=3,                  # node group size
                voltage=0.6,             # bulk parameter assignment
                leak=[0.5, 0.4, 0.3])    # individual parameter assignment

conn = EdgeGroup(source, target,         # source and target node sets
                 PSP(delay=2.0),         # model and parameter defaults
                 edges=[(0, 1), (1, 2),  # list of edge tuples
                        (2, 0)],         # locally indexed w.r.t node sets
                 weight=[1.0, 2.0, 3.0]) # per-edge parameter assignment

pop[1].voltage = 0.8         # linear-based indexing node parameter assignment
conn[(1, 2)].delay = 3.0     # tuple-based indexing edge parameter assignment
```

COMPOSITIONAL MENTAL MODEL

- **Deferred composition**: provide alias classes (analogous to symlinks) for a layer of indirection and providing **pass-by-reference** dependencies for constructing complex network topologies
  - **`NodePort`**: an alias class pointing to set of (external) nodes (e.g. placeholder nodes for network inputs)
  - **`NodeList`**: an alias class with a list of (pointers to) nodes (e.g. a collection of nodes for network outputs)

- Sango also allows for **lists of network components** to be assigned

COMPARING NODE GROUPS, PORTS, AND LISTS



Node ports are alias lists (that contain alias nodes), that may be bound to node groups, node lists, or other node ports

Node lists are concrete lists that contain alias nodes, which can come from multiple node groups, node ports, or other node lists

In this example, during flattening, alias node `D[0]` links to concrete node `A[0]`

DEFINING NETWORKS

- Network definition proceeds simply through creating network objects and **assigning network components** to them
  - We can also wrap/subclass network components (including nesting subnetworks) to create custom network classes

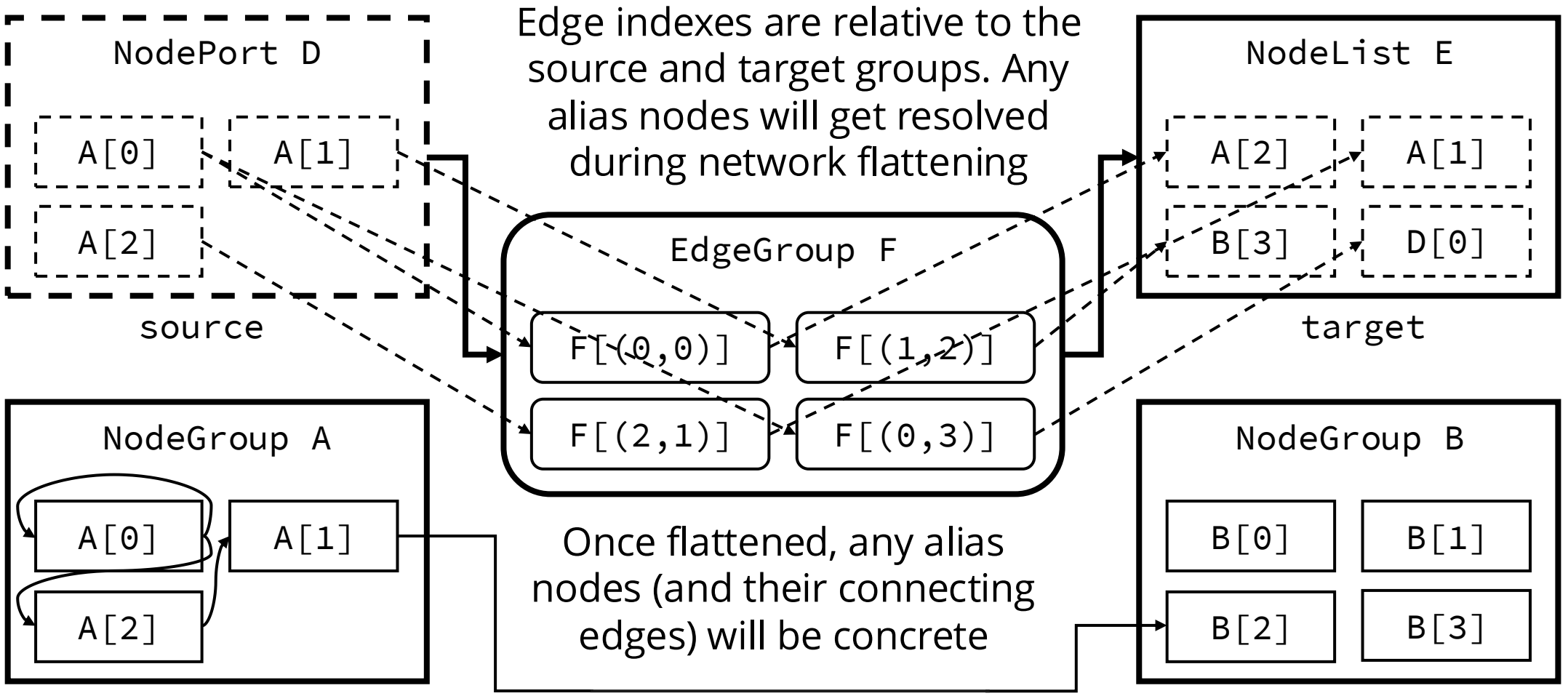
- Two main method definitions for a **`Network`**:
  - **`__init__()`**: this is where placeholder node ports and any relevant network parameters are provided
  - **`build()`**: this is where network components are assigned (allows for procedural construction)

NETWORK CLASS SYNTAX

```
class Linear(Network):
    def __init__(self, size=256):
        super().__init__()      # base class initialization (required)
        self.size = size        # user-defined network parameters
        self.inp = NodePort()   # unsized node port (generates dependency)
        self.ctrl = NodePort(1) # sized node port (no dependency)

    def build(self):
        # Layers (using supplied parameters)
        self.layer = NodeGroup(LIF(), self.size)

        # Procedurally generating edges (using resolved port information)
        edges = [(i,j) for i,j in itertools.product(
            range(self.inp.size),range(self.layer.size))]

        # Connections (using node port placeholder)
        self.dense = EdgeGroup(self.inp, self.layer, PSP(), edges=edges)

        return
```

ADVANCED EDGE CONNECTIONS

Edge groups can also connect between node ports and node lists



BUILDING NETWORKS

- Network construction is performed by calling the top-level **`build()`**
  - This iteratively and **recursively builds** any child networks in the defined hierarchy, and instantiates the network components
  - This process implicitly performs a **topological sort** to resolve any dependencies from alias components (e.g. node ports)

- Network components get assigned a **full path name** when built
  - This uses simple dot (attributes) and bracket notation (lists)
  - This combination of component names, dots, brackets, and indices is **human-readable and unique** (e.g. `net.ff[1].layer[3]`)

NETWORK COMPOSITION SYNTAX

```
net = Network()                    # instantiate network
net.inp = NodeGroup(IN(), 12,      # add an input node group
                    times=...)     # spike times for inputs
net.ff = [Linear(32),              # add a list of networks
          Linear(10)]
# bind network components to ports (using dot notation)
net.bind(net.inp, net.ff[0].inp)
net.bind(net.ff[0].layer, net.ff[1].inp)
net.build()        # build network topology
print(net)         # print the network
```

```
(node) inp
(port) ff[0].inp <- (node) inp
(port) ff[0].ctrl <- (no link)
(node) ff[0].layer
(edge) ff[0].dense: (port) ff[0].inp <- (node) inp -> (node) ff[0].layer
(port) ff[1].inp <- (node) ff[0].layer
(port) ff[1].ctrl <- (no link)
(node) ff[1].layer
(edge) ff[1].dense: (port) ff[1].inp <- (node) ff[0].layer -> (node) ff[1].layer
```

SIMULATING NETWORKS

- The constructed Sango network may be exported **`to_nx()`**
  - This is a **`NetworkX DiGraph`** of nodes and edges with full path names
  - Simulation backends use this to **`compile()`** and **`run()`** the network

- Sango currently interfaces with a variety of neuromorphic tools:
  - **Fugu** – Python framework for computational neural graphs
  - **Brian 2** – Accessible general purpose SNN simulator in Python
  - **STACS** – Large-scale SNN simulator in C++/Charm++, **distributed IR**
  - **Loihi 2** – Scalable digital neuromorphic platform from Intel

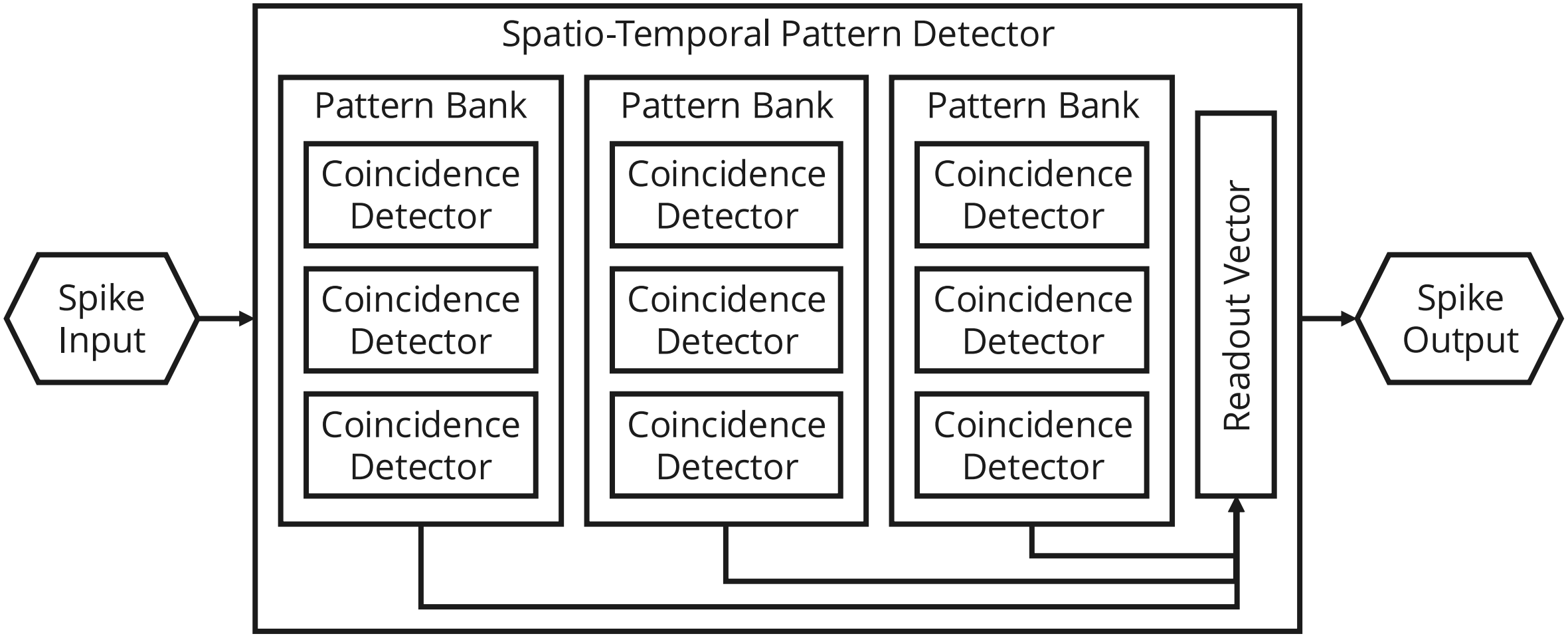
NETWORK SIMULATION SYNTAX

```
# Import the desired simulator backend
from sango.backend import SimBrian

sim = SimBrian(net)  # Pass the built network to the backend translation layer
sim.compile()        # Convert the network onto the backend execution model
sim.run(10.0)        # Simulate the network (arguments may be backend-specific)

spike_list = sim.get_spikes()                  # Get the spike list for each node
node_index = sim.node_map['ff[1].layer[3]']    # Find the index of a node by name
node_spikes = spike_list[node_index]           # Extract the node spike times

# Plot the spike raster (this uses matplotlib's eventplot)
sim.plot_spikes(tick_names=True)
```

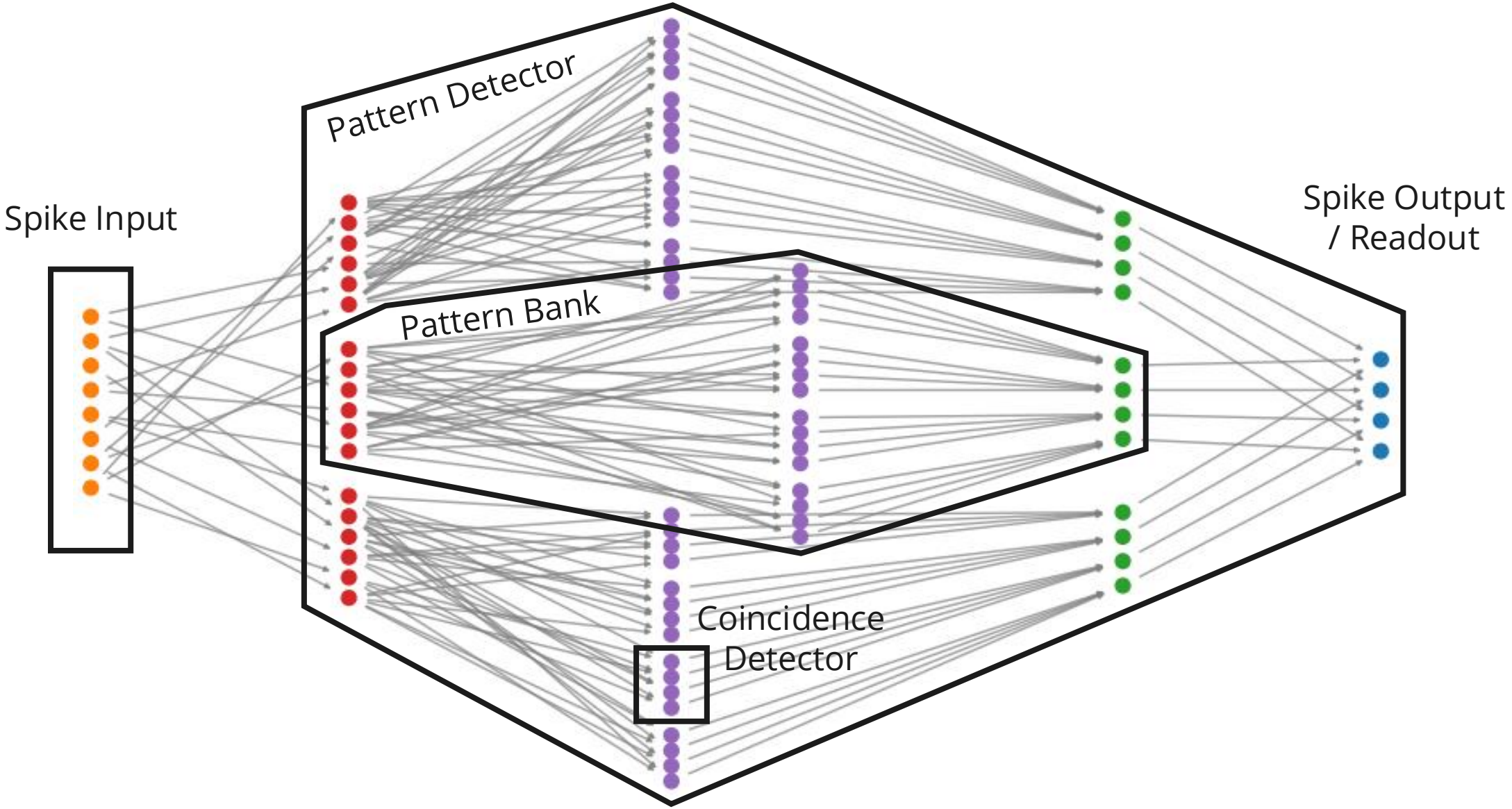
- Sango frontend describes network topology, backends implement dynamics

HIERARCHICAL NETWORK EXAMPLE

- We look at network performing spatio-temporal pattern detection



HIERARCHICAL NETWORK STRUCTURE



HIERARCHICAL NETWORK SYNTAX (1)

```
# Detector over multiple coincidence patterns
class CoincidenceDetector(Network):
    . . .

    def build(self):
        self.detector = NodeGroup(
            LIF(threshold=(self.len_patterns-0.1)), self.num_patterns)
    . . .

# A collection of coincidence detectors
class PatternBank(Network):
    . . .

    def build(self):
        self.cd = [CoincidenceDetector(num_patterns=self.patterns_per_detector,
                                       len_patterns=self.len_patterns)
                   for _ in range(self.num_detectors)]
    . . .
```

HIERARCHICAL NETWORK SYNTAX (2)

```
# A collection of pattern banks, with aggregate readout vector
class PatternDetector(Network):
    . . .

    def build(self):
        self.bank = [PatternBank(num_detectors=self.detectors_per_bank,
                                 patterns_per_detector=self.patterns_per_detector,
                                 len_patterns=self.len_patterns)
                     for _ in range(self.num_banks)]

        # Readout vector performs threshold on pattern banks
        self.readout = NodeGroup(
            LIF(threshold=self.readout_threshold-0.1), self.detectors_per_bank)
    . . .
```

SUMMARY

- We developed a compositional SNN domain-specific language
  - Provides **high-level abstractions** for constructing complex networks
    - Basic classes with **filesystem hierarchy**: `Network`, `NodeGroup`, `EdgeGroup`
    - Alias classes enabling **deferred composition**: `NodePort`, `NodeList`
  - Flattened graph is straightforward to translate and debug
  - We plan on integrating it with more neuromorphic tools
    - Currently interfaces with Fugu, Brian 2, STACS, Loihi 2
- We hope you would be interested in using it!

`https://github.com/sandialabs/Sango`